

Optimizacion Paralela del Sistema de Scoring Metagenomico para Clasificacion Binaria

Proyecto de Computacion de Alto Rendimiento (HPC)

Python · C/OpenMP · MPI · CUDA

Area: Bioinformatica Computacional

Fecha: 7 de mayo de 2026

0. Índice

1. Descripcion del Problema	2
2. Modelo Matematico	2
2.1. Score por Item	2
2.2. Score por Muestra	2
2.3. Funcion Objetivo	3
2.4. Validacion del Scoring	3
2.5. Espacio de Busqueda	3
3. Arquitectura del Sistema	3
3.1. Nivel 1 — Python: Baseline	3
3.2. Nivel 2 — C con OpenMP y MPI	4
3.2.1. C + OpenMP (Memoria Compartida)	4
3.2.2. C + MPI (Memoria Distribuida)	4
3.3. Nivel 3 — CUDA: Aceleracion GPU	4
4. Metricas de Evaluacion	4
4.1. Tiempo de Ejecucion	4
4.2. Speedup	5
4.3. Eficiencia	5
4.4. Ley de Amdahl	5
4.5. Tabla Comparativa	5
5. Estructura del Repositorio	5
6. Script de Generacion de Datos	6
7. Entregables	6

1. Descripcion del Problema

Se busca desarrollar un sistema de optimizacion de alto rendimiento para determinar un vector de pesos $\mathbf{W}$ que maximice el area bajo la curva ROC (AUC) en la clasificacion binaria de muestras biologicas de tipo metagenomico.

El conjunto de datos consiste en **10 muestras** divididas simetricamente:

- **5 muestras** — poblacion *sana* ($y = 0$).
- **5 muestras** — poblacion *enferma* ($y = 1$).

Un scoring bien calibrado producira una distribucion de scores donde los dos grupos sean claramente distinguibles. Si la mayoria de scores altos se concentran en las muestras enfermas (o viceversa de modo consistente), el modelo es valido; en caso contrario, la combinacion de pesos $\mathbf{W}$ no representa adecuadamente los datos.

2. Modelo Matematico

Cada muestra j se describe mediante N items (genomas o taxones). Para cada item i se dispone de tres perfiles:

Cuadro 1: Perfiles por item

Simbolo	Perfil	Descripcion
T_i	Taxonomico	Abundancia relativa de microorganismos.
S_i	Ecológico poblacional	Variables contextuales no genómicas de la muestra.
F_i	Funcional	Presencia/ausencia de genes de interes (beneficos, de resistencia, etc.).

2.1. Score por Item

$$P_i = W_1 T_i + W_2 S_i + W_3 F_i \quad (1)$$

donde $\mathbf{W} = (W_1, W_2, W_3)^\top$ es el vector de pesos a optimizar.

2.2. Score por Muestra

$$\text{Score} = \mathbf{A} \cdot \mathbf{P} \quad (2)$$

- $\mathbf{A} \in \mathbb{R}^{10 \times N}$: **matriz de contribucion** (dimensiones fijas). La entrada a_{ji} pondera el item i en la muestra j .

- $\mathbf{P} \in \mathbb{R}^N$: vector de scores por item (ec. 1).
- $\mathbf{Score} \in \mathbb{R}^{10}$: un score escalar por muestra.

2.3. Funcion Objetivo

$$\max_{\mathbf{W}} \text{AUC}(\mathbf{y}, \mathbf{Score}(\mathbf{W})) \quad (3)$$

El AUC mide la probabilidad de que una muestra enferma reciba score mayor que una sana; $\text{AUC} \in [0,5, 1,0]$ es el rango de interes practico.

2.4. Validacion del Scoring

Para un umbral de decision θ :

$$\text{Consistencia} = \frac{|\{j : \text{Score}_j > \theta, y_j = 1\}|}{5} + \frac{|\{j : \text{Score}_j \leq \theta, y_j = 0\}|}{5} \quad (4)$$

Se considera satisfactorio $\text{Consistencia} \geq 0,8$.

2.5. Espacio de Busqueda

$$W_1 + W_2 + W_3 = 1, \quad W_i \geq 0 \quad (5)$$

3. Arquitectura del Sistema

Cuadro 2: Niveles de implementacion

Nivel	Tecnologia	Paradigma	Proposito
1	Python (NumPy / multiprocessing)	Secuencial / Multicore	Baseline y validacion
2	C + OpenMP C + MPI	Mem. compartida / Distribuida	Paralelismo de CPU
3	PyCUDA / CUDA C	GPU (SIMD masivo)	Aceleracion maxima

3.1. Nivel 1 — Python: Baseline

Librerias: NumPy, scikit-learn, multiprocessing.

Estrategia: *Random Search* sobre K candidatos de $\mathbf{W}$ en el simplex.

Flujo:

1. Cargar A y y .
2. Muestrear K vectores $\mathbf{W}$ sobre el simplex.
3. Para cada $\mathbf{W}$: $\mathbf{P} \rightarrow \text{Score} = A\mathbf{P} \rightarrow \text{AUC}$.
4. Retornar $\mathbf{W}^* = \arg \max \text{AUC}$.
5. Version multicore: repartir los K candidatos entre `cpu_count()` procesos.

3.2. Nivel 2 — C con OpenMP y MPI

3.2.1. C + OpenMP (Memoria Compartida)

- Multiplicacion $A \cdot \mathbf{P}$ con `#pragma omp parallel for`.
- Actualizacion del AUC maximo con `#pragma omp critical`.
- Reduccion de acumuladores con clausula `reduction(+:acum)`.

3.2.2. C + MPI (Memoria Distribuida)

- Proceso *root* genera los K candidatos y distribuye con `MPI_Scatter`.
- Cada proceso evalua su subconjunto de forma independiente.
- Recoleccion del optimo con `MPI_Reduce` (op. `MPI_MAX`).

3.3. Nivel 3 — CUDA: Aceleracion GPU

- Cada hilo evalua un candidato $\mathbf{W}_k$.
- Kernel usa **memoria compartida** para cachear filas de A por bloque.
- Transferencias Host-to-Device realizadas una unica vez antes del kernel.
- Reduccion del AUC con *reduction kernel* estandar.

$$\text{Grid} = \left\lceil \frac{K}{\text{BLOCK_SIZE}} \right\rceil, \quad \text{BLOCK_SIZE} = 256 \quad (6)$$

4. Metricas de Evaluacion

4.1. Tiempo de Ejecucion

$$T = t_{\text{fin}} - t_{\text{inicio}} \quad (7)$$

Medido desde el inicio de la busqueda hasta obtener $\mathbf{W}^*$ (excluye carga de datos).

4.2. Speedup

$$S = \frac{T_{\text{Python}}}{T_{\text{impl}}} \quad (8)$$

4.3. Eficiencia

$$E = \frac{S}{P} \quad (9)$$

donde P es el numero de nucleos/hilos. $E = 1$ indica escalabilidad ideal.

4.4. Ley de Amdahl

$$S_{\text{máx}} = \frac{1}{(1 - f) + \frac{f}{P}} \quad (10)$$

f se estima empiricamente como la fraccion del tiempo total que ocupa la parte paralela.

4.5. Tabla Comparativa

Cuadro 3: Resumen de desempenyo (completar con resultados)

Implementacion	T (s)	S	E	AUC
Python secuencial	—	1.00	—	—
Python multicore	—	—	—	—
C + OpenMP	—	—	—	—
C + MPI	—	—	—	—
CUDA	—	—	—	—

5. Estructura del Repositorio

Listing 1: Organizacion de carpetas

```

1 scoring_metagenomico/
2 |-- data/
3 |   |-- generate_data.py
4 |   |-- matrix_A.npy
5 |   '-- labels.npy
6 |-- python/
7 |   |-- sequential.py
8 |   '-- multicore.py

```

```

9 | -- C_OpenMP_MPI/
10 | | -- scoring_openmp.c
11 | | -- scoring_mpi.c
12 | | -- Makefile
13 | -- CUDA/
14 | | -- scoring_kernel.cu
15 | | -- scoring_pycuda.py
16 | | -- Makefile
17 | -- results/
18 | | -- benchmark.csv
19 | | -- plots/
20 | -- report/
21 | | -- informe_tecnico.pdf

```

6. Script de Generacion de Datos

Listing 2: data/generate_data.py

```

1 import numpy as np
2
3 def generate_data(n_items: int = 50, seed: int = 42):
4     """
5     Genera A en  $R^{10 \times n\_items}$  y etiquetas y.
6     Filas 0-4: sanas (y=0). Filas 5-9: enfermas (y=1).
7     Cada fila de A es Dirichlet(1,...,1) -> suma 1.
8     """
9     rng = np.random.default_rng(seed)
10    A = rng.dirichlet(np.ones(n_items), size=10).astype(np.float32)
11    y = np.array([0]*5 + [1]*5, dtype=np.int32)
12    return A, y
13
14 if __name__ == "__main__":
15    A, y = generate_data(n_items=50)
16    np.save("data/matrix_A.npy", A)
17    np.save("data/labels.npy", y)
18    print(f"A: {A.shape} | y: {y}")

```

7. Entregables

1. **Codigo fuente** organizado en python/, C_OpenMP_MPI/ y CUDA/.
2. **Script de generacion de datos** con N configurable para pruebas de escalabilidad.
3. **Script de benchmark** (run_all.sh) que genera results/benchmark.csv.
4. **Informe tecnico** en PDF con:

- Estrategias de sincronizacion por nivel.
- Analisis de gestion de memoria.
- Graficas comparativas de Speedup y Eficiencia vs. P .
- Analisis de Amdahl con f empirico.
- Discusion de separabilidad de grupos mediante el score optimo.